

DigiGuide : Project Analysis & Developer Guide

AI-Powered Smart Landmark Recognition & Audio Companion | Architecture & Technical Manual

Repository: MakrandGhodke/digiguide	Maintainer: Makrand Ghodke	Date: October 2026
Location Focus: Darmstadt, Germany	Tech Stack: Flutter + FastAPI + CLIP + FAISS	Status: Active Development

1. Executive Summary & Core Mission

DigiGuide is an intelligent, camera-first mobile tourist companion application designed to bring historical landmarks to life through real-time artificial intelligence. Designed specifically for the city of **Darmstadt, Germany**, DigiGuide empowers tourists and visitors to simply point their smartphone camera at any historic monument or architectural site and immediately unlock rich cultural insights, verified fun facts, and hands-free spoken audio guides.

Rather than relying on basic image classification or cloud reverse image search, DigiGuide utilizes an end-to-end **multimodal AI pipeline**: input photos are transformed into dense semantic embeddings using **OpenAI's CLIP** (Contrastive Language-Image Pretraining), queried against a high-speed **FAISS vector database** with multi-angle rotation invariance, and cross-validated against **real-time GPS geofencing**.

Key User Experience Capabilities:

- **Instant Camera Recognition:** Sub-second identification of monuments even under varied lighting, angles, and camera orientations.
- **Curated Cultural Narratives:** Comprehensive historical dossiers, architectural background, and verified fun facts for each landmark.
- **Proximity Audio Tours:** An automated hands-free guide mode that detects when visitors walk within 100 meters of a landmark and speaks natural narration via Text-to-Speech.
- **Live Weather Intelligence:** Seamlessly pulls live temperature and outdoor visiting recommendations using Open-Meteo.
- **Exploration & Personal History:** Categorized landmark discovery (Urban, History, Nature), bookmarking, and local scan history tracking.

Architectural Principle: Clean decoupling between the cross-platform mobile client (Flutter) and the AI inference engine (FastAPI). The backend acts as a high-throughput stateless inference server, while the mobile app handles sensor orchestration (Camera, GPS, TTS) and offline caching for optimal battery and data efficiency.

2. System Workflow & Data Pipeline

When a user captures a photograph in the mobile app, the system executes an optimized 7-stage recognition pipeline:

Step	Component	Action & Processing Details
1. Capture	Flutter Client	User captures image via live viewfinder or gallery. Geolocator simultaneously queries current GPS coordinates (lat, lon).
2. Transport	REST API	Client sends multipart/form-data request to POST /predict containing image bytes and optional latitude/longitude.
3. Rotation Fix	Pillow Engine	Applies ImageOps.exif_transpose() to correct phone sensor orientation, and evaluates 4 angles (0°, 90°, 180°, 270°) to prevent rotated photo failures.
4. Embedding	OpenAI CLIP	Extracts visual semantics into an L2-normalized 512-dimensional vector using openai/clip-vit-base-patch32.
5. Vector Search	FAISS Index	Queries IndexFlatL2 for top-k nearest neighbors. L2 Euclidean distance is converted into cosine similarity: $sim = \max(0, 1 - d^2/2)$.
6. Validation	Geofencing	If similarity ≥ 0.85 , visual match is confirmed. If below threshold and GPS is provided, Haversine formula ranks closest landmarks within 50km as suggestions.
7. Narration	Flutter TTS	Client displays full historical narrative, bullet points, weather, and plays natural 0.5x speed audio narration.

3. Technology Stack Breakdown

Layer	Technology	Role / Implementation Purpose
Mobile UI	Flutter 3.x / Dart	Cross-platform mobile client with custom Material 3 light/dark responsive themes.
State Mgmt	Provider 6.0	ChangeNotifier providers for Favorites, ThemeMode, and background AudioTourService.
Audio / Speech	flutter_tts	Text-to-speech engine delivering clear historical narratives and walking tours.
Sensors / GPS	geolocator & camera	Live camera viewfinder access and continuous GPS stream tracking with 10-meter distance filtering.
Backend API	FastAPI & Uvicorn	Asynchronous ASGI web service supporting multipart uploads and JSON endpoints.
Vision Model	CLIP (HuggingFace)	Pretrained clip-vit-base-patch32 PyTorch model generating 512-dim visual embeddings.
Vector Index	FAISS (faiss-cpu)	High-performance nearest neighbor vector search engine supporting exact Euclidean distance matching.
Authentication	bcrypt & python-jose	Salted SHA-256 password hashing and JWT token issuance for 30-day mobile sessions.
Weather API	Open-Meteo REST	Public weather forecast API mapping WMO weather codes to temperature, icons, and conditions.

4. Repository Structure & Key Components

The project codebase is organized into modular services and data stores:

File / Directory	Component	Responsibility & Key Functions
main.py	FastAPI Server	Entry point for backend server; registers endpoints (/predict, /landmarks, /nearby, /weather, /auth) and contains landmark dossiers.
utils.py	Model Pipeline	Loads CLIP model to GPU/CPU; provides image_to_embedding() with L2 normalization.
build_index.py	Indexer Script	Scans dataset/ photos, feeds them through CLIP, and outputs vector_store.index & metadata.json.
locations.json	Geo Data	Accurate latitude and longitude coordinates for all indexed Darmstadt landmarks.
metadata.json	Vector Mapping	Maps FAISS numeric index IDs to landmark names, filenames, and GPS coordinates.
update_ip.py	Dev Utility	Detects host machine IP and updates frontend/lib/config.dart for USB reverse tunneling or Wi-Fi.
dataset/	Image Dataset	Reference photographic dataset organized into 11 landmark class folders.
frontend/lib/	Flutter Source	Application entry (main.dart), screens (Home, Camera, Explore, Result, Profile), and services.

5. Current Landmark Coverage (11 Darmstadt Sites)

The visual vector store and knowledge base currently index 11 major sites across Darmstadt:

Landmark Slug	Official Name	Category	Coordinates
darmstadium	Darmstadtium Congress Center	Urban / Modern	49.8732° N, 8.6558° E
mathildenhöhe	Mathildenhöhe (UNESCO Site)	History / Art	49.8775° N, 8.6675° E
schloss	Residenzschloss Darmstadt	History / Palace	49.8728° N, 8.6552° E
waldspirale	Waldspirale (Hundertwasser)	Architecture	49.8863° N, 8.6554° E
Luisenplatz	Luisenplatz & Ludwigsmonument	Urban / Center	49.8724° N, 8.6511° E
Hessisches-Landesmuseum	Hessian State Museum	Culture / Museum	49.8745° N, 8.6548° E
Staatstheater	Staatstheater Darmstadt	Culture / Theater	49.8688° N, 8.6516° E
herrengarten	Herrngarten (Central Park)	Nature / Park	49.8760° N, 8.6530° E
orangerie_park	Orangerie Baroque Garden	Nature / Baroque	49.8596° N, 8.6565° E
prinz-emil-garten	Prinz-Emil-Garten	Nature / Park	49.8580° N, 8.6480° E
jugendstilbad	Jugendstilbad (Art Nouveau Spa)	History / Wellness	49.8727° N, 8.6601° E

6. Critical Codebase Findings & Defects to Address

A rigorous forensic review of the repository revealed 4 architectural bugs and technical debt items:

Defect 1: Duplicate & Shadowed Endpoints in main.py

Lines 917 to 1361 are an unmerged duplicate of lines 347 to 881 resulting from a prior Git merge conflict. In FastAPI/Starlette, routes are evaluated in first-registered order. As a result, the newer bug fixes placed at the bottom (specifically the updated Open-Meteo API query format) are never reached, and the older handler intercepts traffic first.

Remediation: Delete the duplicate bottom block (lines 917–1361) and port over any newer query fixes to the top handlers.

Defect 2: Missing POST /feedback Endpoint on Backend

In landmark_result_screen.dart, the mobile client issues a POST request to /feedback whenever a user submits a thumbs up/down accuracy rating. While the database table reviews exists in users.db and an export script (export_reviews.py) expects it, the actual endpoint was never created in main.py, causing mobile review submissions to throw 404 Not Found.

Remediation: Implement `@app.post('/feedback')` to insert feedback records into SQLite.

Defect 3: Split User Database Architecture

Authentication endpoints in main.py read and write user credentials to a flat JSON file (users.json). Concurrently, a relational SQLite database (users.db) with users and reviews schemas exists alongside it. Plaintext JSON storage lacks ACID guarantees and concurrent write locking.

Remediation: Migrate authentication storage from users.json into users.db using standard SQLite queries.

Defect 4: Non-Persistent Favorites in Flutter

In frontend/lib/providers/favorites_provider.dart, favorites are stored in a volatile Dart list (`final List<Landmark> _favorites = []`) without local serialization. When the mobile app is terminated, all user bookmarks are lost. Unlike RecentHistoryService which persists to SharedPreferences, favorites need persistence.

Remediation: Implement JSON serialization and disk storage in FavoritesProvider via SharedPreferences.

7. Developer Setup & Execution Guide

Follow this sequential guide to run both the backend inference service and mobile app:

Step 1: Backend Setup & Launch

```
# 1. Create and activate Python virtual environment
python -m venv .venv
.\.venv\Scripts\activate

# 2. Install PyTorch, Transformers, FAISS, and FastAPI
pip install -r requirements.txt

# 3. Run FastAPI backend server with hot-reload
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Step 2: Network Bridging (Mobile Connection)

```
# Option A: Connected via USB Cable (Fast & Reliable)
python update_ip.py --localhost
adb reverse tcp:8000 tcp:8000

# Option B: Wireless Mode (Phone and PC on same Wi-Fi)
python update_ip.py
```

Step 3: Run Mobile Application

```
cd frontend
flutter pub get
flutter run
```

8. Prioritized Development Roadmap

Phase	Milestone Target	Key Deliverables & Action Items
Phase 1 Stability & Fixes	Codebase Deduplication & Storage	- Remove duplicate routes (lines 917–1361) in main.py. - Add POST /feedback endpoint saving ratings into SQLite. - Unify auth storage into SQLite database. - Persist user favorites to SharedPreferences.
Phase 2 Enhanced UX	Maps & Multilingual Tour	- Interactive Map view (Mapbox / Google Maps) with pins. - German / English bilingual audio guide toggle. - Offline caching for offline tourists without data roaming.
Phase 3 AI & Scale	Model Calibration & Cities	- Calibrate threshold with hybrid visual + GPS scoring. - Expand dataset to Frankfurt, Heidelberg, and Rhine-Neckar. - Curator web dashboard to upload new landmarks.